

Not all predicates must be regular

**Document number:**

P3734R0

Date:

2025-06-14

Audience:

SG9

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3734/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/irregular-pred.cow



ABSTRACT: Some algorithms, such as `std::remove_if` and `std::copy_if` are over-constrained by requiring the predicate to be regular, despite the predicate visiting elements exactly once, making the requirement meaningless. These constraints should be relaxed.

§ Contents

- 1 **Introduction**
 - 1.1 Why these functions are over-constrained
 - 1.2 Possible implementation
- 2 **Motivation**
- 3 **Design discussion**
 - 3.1 But it's bad practice to modify the range in `find_if`!
 - 3.2 But algorithms are too under-specified to permit modification!
 - 3.3 But this makes algorithms harder to reason about!
- 4 **Impact on implementations**
- 5 **Impact on existing code**
- 6 **Impact on the standard**



7	Wording
7.1	[algorithms.requirements]
7.2	[indirectcallable.indirectinvocable]
7.3	[alg.eq.ind.cmp]
7.4	[alg.all.of]
7.5	[alg.any.of]
7.6	[alg.none.of]
7.7	[alg.find]
7.8	[alg.find.last]
7.9	[alg.count]
7.10	[alg.mismatch]
7.11	[alg.equal]
7.12	[alg.starts.with]
7.13	[alg.ends.with]
7.14	[alg.copy]
7.15	[alg.replace]
7.16	[alg.remove]
7.17	[alg.partitions]
8	References

§ 1. Introduction

Consider the following function, which transform a string such as "abc //def" into "abc-def":



EXAMPLE:

```
bool allowed_in_id(char c) {
    return c >= 'a' && c <= 'z';
}

void sanitize_id(std::string& id) {
    auto removed_range = std::ranges::remove_if(id, [](char& c) {
        if (c == ' ') {
            c = '-';
            return false;
        }
        return not allowed_in_id(c);
    });
    id.erase(removed_range.begin(), removed_range.end());
}
```

Such code is perfectly reasonable; it allows us to transform a range and remove some of its elements in a single pass, which may be substantially faster than two separate passes.

While this code works as expected in practice, it *technically* has undefined behavior due to the fact that the predicate passed to `erase_if` violates semantic requirements for the predicate, specified in [\[concept.regularinvocable\]](#):

```
template<class F, class... Args>
concept regular_invocable = invocable<F, Args...>;
```

The `invoke` function call expression shall be equality-preserving ([\[concepts.equality\]](#)) and shall not modify the function object or the arguments.

For example, when visiting ' ' in the string the first time, it is replaced with '-', and `false` is returned, but no action is taken the second time, and `true` is returned. Thus, both semantic requirements of `regular_invocable` are violated.

NOTE: I deliberately did not use `std::erase_if` because the code would be well-defined based on the technicality that the predicate does not apply any non-constant functions through its arguments ([\[algorithms.requirements\]](#) paragraph 15). These semantic requirements are relevant to `remove_if` rather than `ranges::remove_if`, and the former is used by `erase_if` at the time of writing.

§ 1.1. Why these functions are over-constrained

There is no motivation for the behavior being undefined in this case. Algorithms such as `remove_if` have a specification as follows ([\[alg.remove\]](#)):

```
Complexity: Exactly last - first applications of the corresponding predicate and any projection.
```

This means that `remove_if` can only visit each element a single time anyway. If the element is modified by the predicate or if the result of the predicate changes, that is of no consequence because no element is visited twice.

It is also worth pointing out that while `std::remove_if` is restricted, `std::remove` is not. It would be possible to provide a type for which the expression `x == y` modifies `x` or `y`. To be fair, `std::ranges::remove_if` makes use of the `indirect_binary_predicate` concept and is therefore stricter than its non-ranges counterpart.

§ 1.2. Possible implementation

For illustration purposes, here is a reference implementation of `remove_if` taken from [cppreference](#):

```
template<class ForwardIt, class UnaryPred>
ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPred p)
{
    first = std::find_if(first, last, p);
}
```

```

    if (first != last)
        for (ForwardIt i = first; ++i != last;)
            if (!p(*i))
                *first++ = std::move(*i);
    return first;
}

```



As can be seen, the whole algorithm is a simple linear pass from `first` to `last`. No element is visited twice, so who cares if `p` modifies an element?

§ 2. Motivation

We should relax these requirements because they are of no benefit to the implementation, and they render the behavior of useful code undefined by technicality. A useful example for the modifying algorithms has already been presented in [§1. Introduction](#).

Even in the case of non-modifying algorithms, modification is useful.



EXAMPLE: The following code has undefined behavior, even in a single-threaded environment:

```

struct Resource {
    int data;
    std::mutex m;

    int get() {
        std::scoped_lock lock{m};
        return data;
    }
};

bool contains_nonzero(std::span<Resource> resources) {
    return std::ranges::any_of(resources, [](Resource& r) {
        return r.get() != 0;
    });
}

```

This use of `any_of` may seem innocuous, but it would violate the semantic requirements of [\[concept.predicate\]](#) if the arguments to the function object are modified, and locking a mutex may be a modification.

We can imagine similar examples with result of computations being cached inside `Resource`, etc. Any reasonable user should expect this code to work, rather than having undefined behavior based on some gotcha. With the current restrictions, we make many algorithms useless for a wide variety of use cases. This is antithetical to the purpose of any standard library.

§ 3. Design discussion



While the proposal does no more than getting rid of an unnecessary technicality by which some algorithm uses undefined behavior, it is still a fairly significant change. Notably, it would even allow the range to be modified within non-modifying algorithms like `find_if`, and it's not obvious whether that would be okay or desirable. The following subsections address these concerns.

§ 3.1. But it's bad practice to modify the range in `find_if`!

It could be argued that modifying elements within the predicate is bad practice, at least for algorithms that are non-modifying. I actually agree with this, and don't feel like there is much motivation to allow this. However, there is quite a bit of motivation for allowing modification in algorithms that are already modifying, like `remove_if`.

Also, it is not the job of WG21 to police what is "good practice" and "bad practice" by making things undefined behavior. CppCoreGuidelines and other style guides can police things just fine. Even if the predicate modifies the elements it's called with, that is of no consequence to how `find_if` operates, and by that logic, it shouldn't be undefined.

We should also be cautious of status-quo bias. If `find_if` was well-defined for modifying predicates since C++98, and someone proposed to make it undefined *now* for stylistic or philosophical reasons, they would be laughed out of the room.

§ 3.2. But algorithms are too under-specified to permit modification!

Consider the following example:

```
std::vector<int> v(42, 0);
std::ranges::find_if(v, [](int& i) { i = 1; return true; });
```

`find_if` is simply stated to return the first iterator for which the predicate returns `true`. It does not specify that the algorithm is "lazy" and immediately returns upon a `true` result, and it does not specify in which order the elements are traversed. Therefore, it is unspecified how many `0` elements have been turned into `1`, and in what order. However, this is not a problem for this proposal for the following reasons:

- The issue of algorithms being a bit under-specified is entirely pre-existing. If you captured `&i` in a `vector<int*>` within the predicate, you would also not know how large the vector is after running `find_if`, and would not know what the size of that vector is.
- This under-specification still does not justify undefined behavior. Each element that the predicate was invoked with is modified, exactly as the user wrote. It is strictly better to reason that *some* elements are modified (but we don't know which) than for the behavior to be entirely undefined for no technical benefit.
- To a large extent, this is a pre-existing quality-of-implementation issue. It would be extremely surprising to the user if the code above modified more than one `int`, but it

would also be extremely surprising if the predicate was invoked more than once, with or without that modification. Considering the vast amount of existing C++ code, some of it probably assumes that e.g. `find_if` is lazy already. ☀️

§ 3.3. But this makes algorithms harder to reason about!

Currently, we have the "guarantee" that `find_if` does not modify the range because the algorithm itself doesn't do that, and it would result in undefined behavior if the predicate did that. For instance, this would let you reason that `find_if` is *idempotent*; that is, you can call it multiple times in a row on the same range and obtain the same iterator.

However, the flaw in that reasoning lies in that undefined behavior. It is neither statically nor dynamically ensured by any implementation that predicates do not modify the elements passed into them, so at best, we can hope that users avoid modifying the range because they have read [\[algorithms.requirements\] paragraph 6](#). In reality, there is no strict guarantee at all, only hopes and recommendations.

Furthermore, we do not lose any of these guarantees as long as the predicate does not modify the range. If so, `find_if` remains idempotent, as it has been. We are merely widening the contract of `find_if` at no cost to implementations.

§ 4. Impact on implementations

The only change is that in some places, standard concepts are swapped out for exposition-only concepts which only differ in semantic requirements, or vice-versa. For example, `predicate` is replaced with *`boolean-test`* in some places, and both these concepts ultimately expand to *`boolean-testable`*`<invoke_result_t<F, Args>>`.

Therefore, some minor tweaks to implementation-specific concepts may have to be made for internal documentation purposes, but any existing implementation already complies because semantic requirements of concepts are not checked.

§ 5. Impact on existing code

Code with undefined behavior may become well-defined. 😊

§ 6. Impact on the standard

Only algorithms that apply predicates exactly once per element are affected. This includes both modifying algorithms like `remove_if` and non-modifying algorithms like `find`. For such algorithms, modification of the element cannot possibly alter the course of the algorithm.

There are certain edge cases like `adjacent_find_if` and `unique`. While these don't apply predicates multiple times, they do apply projections more than once, and elements are

revisited, so it's possible that a modification performed by the predicate is unsafe.

Another edge case is `std::partition_point`. Although the predicate is not even applied once per element, if the predicate modifies the range, this could retroactively violate the *Preconditions* that the input range is partitioned.



§ 7. Wording

All changes are relative to [N5008].

§ [algorithms.requirements]

Change [algorithms.requirements] paragraph 7 as follows:



When not otherwise constrained, the Predicate or UnaryTest parameter is used whenever an algorithm expects a function object ([function.objects]) that, when applied to the result of dereferencing the corresponding iterator, returns a value testable as `true`. If an algorithm takes Predicate `pred` or UnaryTest `pred` as its argument and `first` as its iterator argument with value type `T`, the expression `pred(*first)` shall be well-formed and the type `decltype(pred(*first))` shall model *boolean-testable* ([concept.booleantestable]). ~~The~~ If specified with type Predicate, the function object `pred` shall not apply any non-constant function through its argument. ~~Given, and given~~ a glvalue `u` of type (possibly `const`) `T` that designates the same object as `*first`, `pred(u)` shall be a valid expression that is equal to `pred(*first)`.

Change [algorithms.requirements] paragraph 8 as follows:



When not otherwise constrained, the BinaryPredicate or BinaryTest parameter is used whenever an algorithm expects a function object that, when applied to the result of dereferencing two corresponding iterators or to dereferencing an iterator and type `T` when `T` is part of the signature, returns a value testable as `true`. If an algorithm takes BinaryPredicate `binary_pred` or BinaryTest `binary_pred` as its argument and `first1` and `first2` as its iterator arguments with respective value types `T1` and `T2`, the expression `binary_pred(*first1, *first2)` shall be well-formed and the type `decltype(binary_pred(*first1, *first2))` shall model *boolean-testable*. Unless otherwise specified, BinaryPredicate always takes the first iterator's `value_type` as its first argument, that is, in those cases when `T` value is part of the signature, the expression `binary_pred(*first1, value)` shall be well-formed and the type `decltype(binary_pred(*first1, value))` shall model *boolean-testable*. If specified with type BinaryPredicate, `binary_pred` shall not apply any non-constant function through any of its arguments. ~~Given, and given~~ a glvalue `u` of type (possibly `const`) `T1` that designates the same object as `*first1`, and a glvalue `v` of type (possibly `const`) `T2` that designates the same object as `*first2`, `binary_pred(u, *first2)`, `binary_pred(*first1, v)`, and `binary_pred(u, v)` shall each be a valid

expression that is equal to `binary_pred(*first1, *first2)`, and `binary_pred(u, value)` shall be a valid expression that is equal to `binary_pred(*first1, value)`. 

§ [indirectcallable.indirectinvocable]

Change [indirectcallable.indirectinvocable] paragraph 1 as follows:



The indirect callable concepts are used to constrain those algorithms that accept callable objects ([func.def]) as arguments.

```
namespace std {
    [...]

    template<class F, class I>
    concept indirect_unary_predicate =
        indirectly_readable<I> &&
        copy_constructible<F> &&
        predicate<F&, indirect_value_t<I>> &&
        predicate<F&, iter_reference_t<I>>;

    template<class F, class Args...>           // exposition only
    concept boolean_test = boolean_testable<invoke_result_t<F, Args...>>;

    template<class F, class I>                 // exposition only
    concept indirect_unary_test =
        indirectly_readable<I> &&
        copy_constructible<F> &&
        boolean_test<F&, indirect_value_t<I>> &&
        boolean_test<F&, iter_reference_t<I>>;

    template<class F, class I1, class I2>
    concept indirect_binary_predicate =
        indirectly_readable<I1> && indirectly_readable<I2> &&
        copy_constructible<F> &&
        predicate<F&, indirect_value_t<I1>, indirect_value_t<I2>> &&
        predicate<F&, indirect_value_t<I1>, iter_reference_t<I2>> &&
        predicate<F&, iter_reference_t<I1>, indirect_value_t<I2>> &&
        predicate<F&, iter_reference_t<I1>, iter_reference_t<I2>>;

    template<class F, class I1, class I2>       // exposition only
    concept indirect_binary_test =
        indirectly_readable<I1> && indirectly_readable<I2> &&
        copy_constructible<F> &&
        boolean_test<F&, indirect_value_t<I1>, indirect_value_t<I2>> &&
        boolean_test<F&, indirect_value_t<I1>, iter_reference_t<I2>> &&
        boolean_test<F&, iter_reference_t<I1>, indirect_value_t<I2>> &&
        boolean_test<F&, iter_reference_t<I1>, iter_reference_t<I2>>;

    [...]
}
```


[Note: *boolean-test* and *predicate* are satisfied (but not modeled) by the same types. — end note]



§ [alg.eq.ind.cmp]

Change [alg.req.ind.cmp] paragraph 1 as follows:



The `indirectly_comparable` concept specifies the common requirements of algorithms that compare values from two different sequences.

```
template<class I1, class I2, class R, class P1 = identity,
        class P2 = identity>
concept indirectly_comparable =
    indirect_binary_predicate<R, projected<I1, P1>, projected<I2, P2>>;

template<class I1, class I2, class R, class P1 = identity, // exposition
        class P2 = identity>
concept indirectly_binary_testable =
    indirect_binary_test<R, projected<I1, P1>, projected<I2, P2>>;
```

[Note: *indirectly-binary-testable* and *indirectly_comparable* are satisfied (but not modeled) by the same types. — end note]

§ [alg.all.of]

Change [alg.all.of] as follows:



```
template<class InputIterator, class Predicate UnaryTest>
constexpr bool all_of(InputIterator first, InputIterator last, Predicate UnaryTest pred);

template<class ExecutionPolicy, class ForwardIterator, class Predicate UnaryTest>
bool all_of(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
            Predicate UnaryTest pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<I, Proj>>
        constexpr bool ranges::all_of(I first, S last, Pred pred, Proj proj = {})
template<input_range R, class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<iterator_1
        constexpr bool ranges::all_of(R&& r, Pred pred, Proj proj = {});
```

1 Let *E* be:

- `pred(*i)` for the overloads in namespace `std`;
- `invoke(pred, invoke(proj, *i))` for the overloads in namespace `ranges`.

2 Returns: `false` if *E* is `false` for some iterator *i* in the range `[first, last)`, and `true` otherwise.

3 *Complexity*: At most last - first applications of the predicate and any projection.



§ [alg.any.of]

Change [alg.any.of] as follows:



```
template<class InputIterator, class Predicate UnaryTest>
constexpr bool any_of(InputIterator first, InputIterator last, Predicate
template<class ExecutionPolicy, class ForwardIterator, class Predicate Uni
bool any_of(ExecutionPolicy&& exec, ForwardIterator first, ForwardItera
Predicate UnaryTest pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<I, Proj>>
constexpr bool ranges::any_of(I first, S last, Pred pred, Proj proj = {})
template<input_range R, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<iterator_1
constexpr bool ranges::any_of(R&& r, Pred pred, Proj proj = {});
```

1 Let E be:

- $\text{pred}(*i)$ for the overloads in namespace `std`;
- $\text{invoke}(\text{pred}, \text{invoke}(\text{proj}, *i))$ for the overloads in namespace `ranges`.

2 *Returns*: `true` if E is `true` for some iterator i in the range $[\text{first}, \text{last})$, and `false` otherwise.

3 *Complexity*: At most last - first applications of the predicate and any projection.

§ [alg.none.of]

Change [alg.none.of] as follows:



```
template<class InputIterator, class Predicate UnaryTest>
constexpr bool none_of(InputIterator first, InputIterator last, Predicate
template<class ExecutionPolicy, class ForwardIterator, class Predicate Uni
bool none_of(ExecutionPolicy&& exec, ForwardIterator first, ForwardItera
Predicate UnaryTest pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<I, Proj>>
constexpr bool ranges::none_of(I first, S last, Pred pred, Proj proj = {}
template<input_range R, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<iterator_1
constexpr bool ranges::none_of(R&& r, Pred pred, Proj proj = {});
```



1 Let E be:

- `pred(*i)` for the overloads in namespace `std`;
- `invoke(pred, invoke(proj, *i))` for the overloads in namespace `ranges`.

2 *Returns*: `false` if E is `true` for some iterator i in the range $[first, last)$, and `true` otherwise.

3 *Complexity*: At most `last - first` applications of the predicate and any projection.

§ [alg.find]

Change `[alg.find]` as follows:



```
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>
constexpr InputIterator find(InputIterator first, InputIterator last,
                             const T& value);
template<class ExecutionPolicy, class ForwardIterator,
         class T = iterator_traits<ForwardIterator>::value_type>
ForwardIterator find(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
                     const T& value);

template<class InputIterator, class Predicate UnaryTest>
constexpr InputIterator find_if(InputIterator first, InputIterator last,
                                Predicate UnaryTest pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator find_if(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
                        Predicate UnaryTest pred);

template<class InputIterator, class Predicate UnaryTest>
constexpr InputIterator find_if_not(InputIterator first, InputIterator last,
                                    Predicate UnaryTest pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate UnaryTest>
ForwardIterator find_if_not(ExecutionPolicy&& exec,
                            ForwardIterator first, ForwardIterator last,
                            Predicate UnaryTest pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate<indirect_binary_test<ranges::equal_to, projected<I, Proj>, const T>,
                                   const T>
constexpr I ranges::find(I first, S last, const T& value, Proj proj = {});
template<input_range R, class Proj = identity, class T = projected_value_t<R, Proj>>
requires indirect_binary_predicate<indirect_binary_test<ranges::equal_to, projected<iterator_t<R>, const T>,
                                   const T>
constexpr borrowed_iterator_t<R>
ranges::find(R&& r, const T& value, Proj proj = {});
template<input_iterator I, sentinel_for<I> S, class Proj = identity,
```



```

indirect_unary_predicate indirect-unary-test<projected<I, Proj>>
constexpr I ranges::find_if(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<iterator_1
constexpr borrowed_iterator_t<R>
ranges::find_if(R&& r, Pred pred, Proj proj = {});
template<input_iterator I, sentinel_for<I> S, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<I, Proj>>
constexpr I ranges::find_if_not(I first, S last, Pred pred, Proj proj =
template<input_range R, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<iterator_1
constexpr borrowed_iterator_t<R>
ranges::find_if_not(R&& r, Pred pred, Proj proj = {});

```

1 Let E be:

- $*i == \text{value}$ for `find`;
- `pred(*i) != false` for `find_if`;
- `pred(*i) == false` for `find_if_not`;
- `bool(invoker(proj, *i) == value)` for `ranges::find`;
- `bool(invoker(pred, invoker(proj, *i)))` for `ranges::find_if`;
- `bool(!invoker(pred, invoker(proj, *i)))` for `ranges::find_if_not`.

2 *Returns*: The first iterator i in the range $[first, last)$ for which E is `true`.
Returns `last` if no such iterator is found.

3 *Complexity*: At most `last - first` applications of the corresponding predicate and any projection.

§ [alg.find.last]

Change [alg.find.last] as follows:



```

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate
indirect-binary-test<ranges::equal_to, projected<I, Proj>, const
constexpr subrange<I> ranges::find_last(I first, S last, const T& value,
template<forward_range R, class Proj = identity,
class T = projected_value_t<iterator_t<R>, Proj>>
requires indirect_binary_predicate
indirect-binary-test<ranges::equal_to, projected<iterator_t<R>,
constexpr borrowed_subrange_t<R> ranges::find_last(R&& r, const T& value
template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<I, Proj>>
constexpr subrange<I> ranges::find_last_if(I first, S last, Pred pred, I
template<forward_range R, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<iterator_1
constexpr borrowed_subrange_t<R> ranges::find_last_if(R&& r, Pred pred,

```

```
template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<I, Proj>>
    constexpr subrange<I> ranges::find_last_if_not(I first, S last, Pred pred, Proj proj)
template<forward_range R, class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<iterator_t<R>, Proj>>
    constexpr borrowed_subrange_t<R> ranges::find_last_if_not(R&& r, Pred pred, Proj proj)
```

1 Let E be:

- `bool(invoker(proj, *i) == value)` for `ranges::find_last`;
- `bool(invoker(pred, invoker(proj, *i)))` for `ranges::find_last_if`;
- `bool(!invoker(pred, invoker(proj, *i)))` for `ranges::find_last_if_not`.

2 Returns: Let i be the last iterator in the range $[first, last)$ for which E is `true`. Returns `{i, last}`, or `{last, last}` if no such iterator is found.

3 Complexity: At most `last - first` applications of the corresponding predicate and projection.

§ [alg.count]

Change [alg.count] as follows:



```
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>
    constexpr typename iterator_traits<InputIterator>::difference_type
        count(InputIterator first, InputIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
    typename iterator_traits<ForwardIterator>::difference_type
        count(ExecutionPolicy&& exec,
              ForwardIterator first, ForwardIterator last, const T& value);

template<class InputIterator, class Predicate UnaryTest>
    constexpr typename iterator_traits<InputIterator>::difference_type
        count_if(InputIterator first, InputIterator last, Predicate UnaryTest, Proj proj)
template<class ExecutionPolicy, class ForwardIterator, class Predicate UnaryTest, Proj proj>
    typename iterator_traits<ForwardIterator>::difference_type
        count_if(ExecutionPolicy&& exec,
              ForwardIterator first, ForwardIterator last, Predicate UnaryTest, Proj proj)

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate indirect_binary_test<ranges::equal_to, projected<I, Proj>, const T&>
    constexpr iter_difference_t<I>
        ranges::count(I first, S last, const T& value, Proj proj = {});
template<input_range R, class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
    requires indirect_binary_predicate indirect_binary_test<ranges::equal_to, projected<iterator_t<R>, Proj>, const T&>
    constexpr iter_difference_t<R>
        ranges::count(R&& r, const T& value, Proj proj = {});
```



```
constexpr range_difference_t<R>
    ranges::count(R&& r, const T& value, Proj proj = {});
template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<I, Proj>>
constexpr iter_difference_t<I>
    ranges::count_if(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<iterator_1
constexpr range_difference_t<R>
    ranges::count_if(R&& r, Pred pred, Proj proj = {});
```

1 Let E be:

- $*i == \text{value}$ for the overloads with no parameter `pred` or `proj`;
- $\text{pred}(*i) != \text{false}$ for the overloads with a parameter `pred` but no parameter `proj`;
- $\text{invoke}(\text{proj}, *i) == \text{value}$ for the overloads with a parameter `proj` but no parameter `pred`;
- $\text{bool}(\text{invoke}(\text{pred}, \text{invoke}(\text{proj}, *i)))$ for the overloads with both parameters `proj` and `pred`.

2 *Effects*: Returns the number of iterators i in the range $[\text{first}, \text{last})$ for which E holds.

3 *Complexity*: Exactly $\text{last} - \text{first}$ applications of the corresponding predicate and any projection.

§ [alg.mismatch]

Change [alg.mismatch] as follows:



```
template<class InputIterator1, class InputIterator2>
    constexpr pair<InputIterator1, InputIterator2>
        mismatch(InputIterator1 first1, InputIterator1 last1,
                 InputIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    pair<ForwardIterator1, ForwardIterator2>
        mismatch(ExecutionPolicy&& exec,
                 ForwardIterator1 first1, ForwardIterator1 last1,
                 ForwardIterator2 first2);

template<class InputIterator1, class InputIterator2,
        class BinaryPredicate BinaryTest>
    constexpr pair<InputIterator1, InputIterator2>
        mismatch(InputIterator1 first1, InputIterator1 last1,
                 InputIterator2 first2, BinaryPredicate BinaryTest pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate BinaryTest>
    pair<ForwardIterator1, ForwardIterator2>
```



```
mismatch(ExecutionPolicy&& exec,
         ForwardIterator1 first1, ForwardIterator1 last1,
         ForwardIterator2 first2, BinaryPredicate BinaryTest pred);

template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair<ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec,
         ForwardIterator1 first1, ForwardIterator1 last1,
         ForwardIterator2 first2, ForwardIterator2 last2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate BinaryTest>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2,
         BinaryPredicate BinaryTest pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair<ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec,
         ForwardIterator1 first1, ForwardIterator1 last1,
         ForwardIterator2 first2, ForwardIterator2 last2,
         BinaryPredicate BinaryTest pred);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr ranges::mismatch_result<I1, I2>
ranges::mismatch(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                 Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr ranges::mismatch_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
ranges::mismatch(R1&& r1, R2&& r2, Pred pred = {},
                 Proj1 proj1 = {}, Proj2 proj2 = {});
```

1 Let $last2$ be $first2 + (last1 - first1)$ for the overloads with no parameter $last2$ or $r2$.

2 Let E be:

- $!(*(first1 + n) == *(first2 + n))$ for the overloads with no parameter $pred$;
- $pred(*(first1 + n), *(first2 + n)) == false$ for the overloads with a parameter $pred$ and no parameter $proj1$;

- `!invoke(pred, invoke(proj1, *(first1 + n)), invoke(proj2, *(first2 + n)))` for the overloads with both parameters `pred` and `proj1`. 

3 Let N be $\min(\text{last1} - \text{first1}, \text{last2} - \text{first2})$.

4 *Returns:* `{ first1 + n, first2 + n }`, where n is the smallest integer in $[0, N)$ such that E holds, or N if no such integer exists.

5 *Complexity:* At most N applications of the corresponding predicate and any projections.

§ [alg.equal]

Change [alg.equal] as follows:



```
template<class InputIterator1, class InputIterator2>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    bool equal(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate BinaryTest>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, BinaryPredicate BinaryTest);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
    bool equal(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    bool equal(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate BinaryTest>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2,
                        BinaryPredicate BinaryTest pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate BinaryTest>
    bool equal(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              BinaryPredicate pred);
```

```

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable indirectly_binary_testable<I1, I2, Pred,
constexpr bool ranges::equal(I1 first1, S1 last1, I2 first2, S2 last2,
                             Pred pred = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable
        indirectly_binary_testable<iterator_t<R1>, iterator_t<R2>, Pred>
constexpr bool ranges::equal(R1&& r1, R2&& r2, Pred pred = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- last2 be first2 + (last1 - first1) for the overloads with no parameter last2 or r2;
- pred be equal_to{ } for the overloads with no parameter pred;
- *E* be:
 - pred(*i, *(first2 + (i - first1))) for the overloads with no parameter proj1;
 - invoke(pred, invoke(proj1, *i), invoke(proj2, *(first2 + (i - first1)))) for the overloads with parameter proj1.

2 Returns: If last1 - first1 != last2 - first2, return false. Otherwise return true if *E* holds for every iterator *i* in the range [first1last1). Otherwise, returns false.

3 Complexity: If

- the types of first1, last1, first2, and last2 meet the Cpp17RandomAccessIterator requirements ([random.access.iterators]) and last1 - first1 != last2 - first2 for the overloads in namespace std;
- the types of first1, last1, first2, and last2 pairwise model sized_sentinel_for ([iterator.concept.sizedsentinel]) and last1 - first1 != last2 - first2 for the first overload in namespace ranges,
- R1 and R2 each model sized_range and ranges::distance(r1) != ranges::distance(r2) for the second overload in namespace ranges,

then no applications of the corresponding predicate and each projection; otherwise,

- For the overloads with no ExecutionPolicy, at most min(last1 - first1, last2 - first2) applications of the corresponding predicate and any projections.

- For the overloads with an ExecutionPolicy, $\mathcal{O}(\min(\text{last1} - \text{first1}, \text{last2} - \text{first2}))$ applications of the corresponding predicate.



§ [alg.starts.with]

Change [alg.starts.with] as follows:



```
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::starts_with(I1 first1, S1 last1, I2 first2, S2 last2, Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ranges::starts_with(R1&& r1, R2&& r2, Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

1 Returns:

```
ranges::mismatch(std::move(first1), last1, std::move(first2), last2, pred, proj1, proj2).in2 == last2
```

§ [alg.ends.with]

Change [alg.ends.with] as follows:



```
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires (forward_iterator<I1> || sized_sentinel_for<S1, I1>) &&
(forward_iterator<I2> || sized_sentinel_for<S2, I2>) &&
indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::ends_with(I1 first1, S1 last1, I2 first2, S2 last2, Proj1 proj1 = {}, Proj2 proj2 = {});
```

1 Let $N1$ be $\text{last1} - \text{first1}$ and $N2$ be $\text{last2} - \text{first2}$.

2 Returns: `false` if $N1 < N2$, otherwise

```
ranges::equal(std::move(first1) + (N1 - N2), last1, std::move(first2), last2, pred, proj1, proj2)
```

```
template<input_range R1, input_range R2, class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires (forward_range<R1> || sized_range<R1>) &&
(forward_range<R2> || sized_range<R2>) &&
indirectly_comparable<R1, R2, Pred, Proj1, Proj2>
constexpr bool ranges::ends_with(R1&& r1, R2&& r2, Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

```

indirectly-binary-testable<iterator_t<R1>, iterator_t<R2>, Pred
constexpr bool ranges::ends_with(R1&& r1, R2&& r2, Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});

```

3 Let N_1 be `ranges::distance(r1)` and N_2 be `ranges::distance(r2)`.

4 *Returns*: `false` if $N_1 < N_2$, otherwise

```

ranges::equal(views::drop(ranges::ref_view(r1), N1 - static_cast<decl
                r2, pred, proj1, proj2)

```

§ [alg.copy]

Change [alg.copy] as follows:



[...]

```

template<class InputIterator, class OutputIterator, class Predicate Unary1
constexpr OutputIterator copy_if(InputIterator first, InputIterator last
                                OutputIterator result, Predicate Unary1
template<class ExecutionPolicy, class ForwardIterator1, class ForwardItera
class Predicate UnaryTest>
ForwardIterator2 copy_if(ExecutionPolicy&& exec,
                        ForwardIterator1 first, ForwardIterator1 last,
                        ForwardIterator2 result, Predicate UnaryTest p1
template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, clas
indirect_unary_predicate indirect-unary-test<projected<I, Proj>>
requires indirectly_copyable<I, O>
constexpr ranges::copy_if_result<I, O>
    ranges::copy_if(I first, S last, O result, Pred pred, Proj proj = {});
template<input_range R, weakly_incrementable O, class Proj = identity,
indirect_unary_predicate indirect-unary-test<projected<iterator_t<R>, O>
requires indirectly_copyable<iterator_t<R>, O>
constexpr ranges::copy_if_result<borrowed_iterator_t<R>, O>
    ranges::copy_if(R&& r, O result, Pred pred, Proj proj = {});

```

15 Let E be:

- `bool(pred(*i))` for the overloads in namespace `std`;
- `bool(invoker(pred, invoker(proj, *i)))` for the overloads in namespace `ranges`,

and N be the number of iterators i in the range `[first, last)` for which the condition E holds.

16 *Preconditions*: The ranges `[first, last)` and `[result, result + (last - first))` do not overlap.

[Note: For the overload with an `ExecutionPolicy`, there might be a performance cost if `iterator_traits<ForwardIterator1>::value_type` is not `Cpp17MoveConstructible` ([cpp17.moveconstructible]). — end note]

17 *Effects*: Copies all of the elements referred to by the iterator `i` in the range `[first, last)` for which `E` is `true`.



18 *Returns*:

- `result + N` for the overloads in namespace `std`.
- `{last, result + N}` for the overloads in namespace `ranges`.

19 *Complexity*: Exactly `last - first` applications of the corresponding predicate and any projection.

20 *Remarks*: Stable (`[algorithm.stable]`).

[...]

§ [alg.replace]

Change `[alg.replace]` as follows:



```
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr void replace(ForwardIterator first, ForwardIterator last,
                       const T& old_value, const T& new_value);

template<class ExecutionPolicy, class ForwardIterator,
         class T = iterator_traits<ForwardIterator>::value_type>
void replace(ExecutionPolicy&& exec,
             ForwardIterator first, ForwardIterator last,
             const T& old_value, const T& new_value);

template<class ForwardIterator, class Predicate UnaryTest,
         class T = iterator_traits<ForwardIterator>::value_type>
constexpr void replace_if(ForwardIterator first, ForwardIterator last,
                          Predicate UnaryTest pred, const T& new_value);

template<class ExecutionPolicy, class ForwardIterator, class Predicate UnaryTest,
         class T = iterator_traits<ForwardIterator>::value_type>
void replace_if(ExecutionPolicy&& exec,
                ForwardIterator first, ForwardIterator last,
                Predicate UnaryTest pred, const T& new_value);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         class T1 = projected_value_t<I, Proj>, class T2 = T1>
requires indirectly_writable<I, const T2&> &&
         indirect_binary_predicate<indirect-binary-test<ranges::equal_to, projected<I, Proj>, const T2&>>
constexpr I
    ranges::replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj);

template<input_range R, class Proj = identity,
         class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = T1>
requires indirectly_writable<iterator_t<R>, const T2&> &&
         indirect_binary_predicate<indirect-binary-test<ranges::equal_to, projected<iterator_t<R>, Proj>, const T2&>>
constexpr borrowed_iterator_t<R>
    ranges::replace(R&& r, const T1& old_value, const T2& new_value, Proj proj);
```



```

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>,
        indirect_unary_predicate indirect_unary_test<projected<I, Proj>>
requires indirectly_writable<I, const T&>
constexpr I ranges::replace_if(I first, S last, Pred pred, const T& new_value) {
template<input_range R, class Proj = identity, class T = projected_value_t<R, Proj>,
        indirect_unary_predicate indirect_unary_test<projected<iterator_t<R>, Proj>>
requires indirectly_writable<iterator_t<R>, const T&>
constexpr borrowed_iterator_t<R>
ranges::replace_if(R&& r, Pred pred, const T& new_value, Proj proj = Proj{}) {

```

1 Let E be

- `bool(*i == old_value)` for `replace`;
- `bool(pred(*i))` for `replace_if`;
- `bool(invoker(proj, *i) == old_value)` for `ranges::replace`;
- `bool(invoker(pred, invoker(proj, *i)))` for `ranges::replace_if`.

2 *Mandates*: `new_value` is writable ([[iterator.requirements.general](#)]) to `first`.

3 *Effects*: Substitutes elements referred by the iterator `i` in the range `[first, last)` with `new_value`, when E is `true`.

4 *Returns*: `last` for the overloads in namespace `ranges`.

5 *Complexity*: Exactly `last - first` applications of the corresponding predicate and any projection.

```

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator
replace_copy(InputIterator first, InputIterator last,
             OutputIterator result,
             const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class T>
replace_copy(ExecutionPolicy&& exec,
             ForwardIterator1 first, ForwardIterator1 last,
             ForwardIterator2 result,
             const T& old_value, const T& new_value);

template<class InputIterator, class OutputIterator, class Predicate UnaryTest>
constexpr OutputIterator
replace_copy_if(InputIterator first, InputIterator last,
               OutputIterator result,
               Predicate UnaryTest pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate UnaryTest, class T>
replace_copy_if(ExecutionPolicy&& exec,
               ForwardIterator1 first, ForwardIterator1 last,
               ForwardIterator2 result,
               Predicate UnaryTest pred, const T& new_value);

```



```
template<input_iterator I, sentinel_for<I> S, class O,
        class Proj = identity, class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<I, Proj>,
        requires indirectly_copyable<I, O> &&
        indirect_binary_predicate<indirect-binary-test<ranges::equal_to, projected<I, Proj>, const T1&, const T2&>
        output_iterator<O, const T2&>
    constexpr ranges::replace_copy_result<I, O>
    ranges::replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value, Proj proj = {});

template<input_range R, class O, class Proj = identity,
        class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = iter_value_t<R, Proj>,
        requires indirectly_copyable<iterator_t<R>, O> &&
        indirect_binary_predicate<indirect-binary-test<ranges::equal_to, projected<iterator_t<R>, const T1&, const T2&>
        && output_iterator<O, const T2&>
    constexpr ranges::replace_copy_result<borrowed_iterator_t<R>, O>
    ranges::replace_copy(R&& r, O result, const T1& old_value, const T2& new_value, Proj proj = {});

template<input_iterator I, sentinel_for<I> S, class O, class T = iter_value_t<I, identity>,
        class Proj = identity,
        indirect_unary_predicate<indirect-unary-test<projected<I, Proj>>, const T&>
    requires indirectly_copyable<I, O> && output_iterator<O, const T&>
    constexpr ranges::replace_copy_if_result<I, O>
    ranges::replace_copy_if(I first, S last, O result, Pred pred, const T& new_value, Proj proj = {});

template<input_range R, class O, class T = iter_value_t<R, identity>, class Proj = identity,
        indirect_unary_predicate<indirect-unary-test<projected<iterator_t<R>, Proj>>, const T&>
    requires indirectly_copyable<iterator_t<R>, O> && output_iterator<O, const T&>
    constexpr ranges::replace_copy_if_result<borrowed_iterator_t<R>, O>
    ranges::replace_copy_if(R&& r, O result, Pred pred, const T& new_value, Proj proj = {});
```

6 Let E be

- `bool(*(first + (i - result)) == old_value)` for `replace_copy`;
- `bool(pred(*(first + (i - result))))` for `replace_copy_if`;
- `bool(invoker(proj, *(first + (i - result))) == old_value)` for `ranges::replace_copy`;
- `bool(invoker(pred, invoker(proj, *(first + (i - result))))` for `ranges::replace_copy_if`.

7 *Mandates*: The results of the expressions `*first` and `new_value` are writable ([iterator.requirements.general]) to result.

8 *Preconditions*: The ranges `[first, last)` and `[result, result + (last - first))` do not overlap.

9 *Effects*: Assigns through every iterator `i` in the range `[result, result + (last - first))` a new corresponding value

- new_value if E is `true` or
- `*(first + (i - result))` otherwise.



10 Returns:

- `result + (last - first)` for the overloads in namespace `std`.
- `{last, result + (last - first)}` for the overloads in namespace `ranges`.

11 Complexity: Exactly `last - first` applications of the corresponding predicate and any projection.



NOTE: It is strange that `replace` is not a stable algorithm, unlike `remove` and `copy`, but that's probably best solved in a different issue/paper.

§ [alg.remove]

Change [alg.remove] as follows:



```
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>
    constexpr ForwardIterator remove(ForwardIterator first, ForwardIterator
                                     const T& value);
template<class ExecutionPolicy, class ForwardIterator,
    class T = iterator_traits<ForwardIterator>::value_type>
    ForwardIterator remove(ExecutionPolicy&& exec,
                          ForwardIterator first, ForwardIterator last,
                          const T& value);

template<class ForwardIterator, class Predicate UnaryTest>
    constexpr ForwardIterator remove_if(ForwardIterator first, ForwardIterator last,
                                       Predicate UnaryTest pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate UnaryTest>
    ForwardIterator remove_if(ExecutionPolicy&& exec,
                             ForwardIterator first, ForwardIterator last,
                             Predicate UnaryTest pred);

template<permutable I, sentinel_for<I> S, class Proj = identity,
    class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate<indirect-binary-test<ranges::equal_to, projected<I, Proj>, const T&>,
    constexpr subrange<I> ranges::remove(I first, S last, const T& value, Proj proj = {});
template<forward_range R, class Proj = identity,
    class T = projected_value_t<iterator_t<R>, Proj>>
    requires permutable<iterator_t<R>> &&
        indirect_binary_predicate<indirect-binary-test<ranges::equal_to, projected<iterator_t<R>, const T&>,
    constexpr borrowed_subrange_t<R> ranges::remove(R&& r, const T& value, Proj proj = {});
template<permutable I, sentinel_for<I> S, class Proj = identity,
    requires indirect_unary_predicate<indirect-unary-test<ranges::equal_to, projected<I, Proj>, const T&>>
```

```

    indirect-unary-test<projected<I, Proj>> Pred>
constexpr subrange<I> ranges::remove_if(I first, S last, Pred pred, Proj proj,
    indirect-unary-predicate
    indirect-unary-test<projected<iterator_t<R>, Proj>> Pred>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R>
ranges::remove_if(R&& r, Pred pred, Proj proj = {});

```

1 Let E be

- `bool(*i == value)` for `remove`;
- `bool(pred(*i))` for `remove_if`;
- `bool(invoker(proj, *i) == value)` for `ranges::remove`;
- `bool(invoker(pred, invoker(proj, *i)))` for `ranges::remove_if`.

2 *Preconditions*: For the algorithms in namespace `std`, the type of `*first` meets the `Cpp17MoveAssignable` requirements ([\[cpp17.moveassignable\]](#)).

3 *Effects*: Eliminates all the elements referred to by iterator `i` in the range `[first, last)` for which E holds.

4 *Returns*: Let j be the end of the resulting range. Returns:

- j for the overloads in namespace `std`.
- `{j, last}` for the overloads in namespace `ranges`.

5 *Complexity*: Exactly `last - first` applications of the corresponding predicate and any projection.

6 *Remarks*: Stable ([\[algorithm.stable\]](#)).

7 [Note: Each element in the range `[ret, last)`, where `ret` is the returned value, has a valid but unspecified state, because the algorithms can eliminate elements by moving from elements that were originally in that range. — end note]

```

template<class InputIterator, class OutputIterator,
        class T = iterator_traits<InputIterator>::value_type>
constexpr OutputIterator
remove_copy(InputIterator first, InputIterator last,
            OutputIterator result, const T& value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class T = iterator_traits<ForwardIterator1>::value_type>
ForwardIterator2
remove_copy(ExecutionPolicy&& exec,
            ForwardIterator1 first, ForwardIterator1 last,
            ForwardIterator2 result, const T& value);

template<class InputIterator, class OutputIterator, class Predicate UnaryTest>
constexpr OutputIterator
remove_copy_if(InputIterator first, InputIterator last,
               OutputIterator result, Predicate UnaryTest pred);

```



```
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate UnaryTest>
ForwardIterator2
remove_copy_if(ExecutionPolicy&& exec,
               ForwardIterator1 first, ForwardIterator1 last,
               ForwardIterator2 result, Predicate UnaryTest pred);

template<input_iterator I, sentinel_for<I> S, weakly_increamentable O,
        class Proj = identity, class T = projected_value_t<I, Proj>>
requires indirectly_copyable<I, O> &&
        indirect_binary_predicate
        indirect-binary-test<ranges::equal_to, projected<I, Proj>, const T&>
constexpr ranges::remove_copy_result<I, O>
ranges::remove_copy(I first, S last, O result, const T& value, Proj proj) {
template<input_range R, weakly_increamentable O, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
requires indirectly_copyable<iterator_t<R>, O> &&
        indirect_binary_predicate
        indirect-binary-test<ranges::equal_to, projected<iterator_t<R>, Proj>, const T&>
constexpr ranges::remove_copy_result<borrowed_iterator_t<R>, O>
ranges::remove_copy(R&& r, O result, const T& value, Proj proj = {});
template<input_iterator I, sentinel_for<I> S, weakly_increamentable O,
        class Proj = identity, indirect_unary_predicate
        indirect-unary-test<projected<I, Proj>, Proj> P>
requires indirectly_copyable<I, O>
constexpr ranges::remove_copy_if_result<I, O>
ranges::remove_copy_if(I first, S last, O result, Pred pred, Proj proj) {
template<input_range R, weakly_increamentable O, class Proj = identity,
        indirect_unary_predicate
        indirect-unary-test<projected<iterator_t<R>, Proj>, Proj> Pred>
requires indirectly_copyable<iterator_t<R>, O>
constexpr ranges::remove_copy_if_result<borrowed_iterator_t<R>, O>
ranges::remove_copy_if(R&& r, O result, Pred pred, Proj proj = {});
```

8 Let E be

- `bool(*i == value)` for `remove_copy`;
- `bool(pred(*i))` for `remove_copy_if`;
- `bool(invoker(proj, *i) == value)` for `ranges::remove_copy`;
- `bool(invoker(pred, invoker(proj, *i)))` for `ranges::remove_copy_if`.

9 Let N be the number of elements in `[first, last)` for which E is `false`.

10 *Mandates*: $\ast$ `first` is writable ([\[iterator.requirements.general\]](#)) to `result`.

11 *Preconditions*: The ranges `[first, last)` and `[result, result + (last - first))` do not overlap.

[Note: For the overloads with an `ExecutionPolicy`, there might be a performance cost if `iterator_traits<ForwardIterator1>::value_type` does not meet the `Cpp17MoveConstructible` ([\[cpp17.moveconstructible\]](#)) requirements. — end note]

12 *Effects*: Copies all the elements referred to by the iterator `i` in the range `[first, last)` for which `E` is `false`.



13 *Returns*:

- `result + N`, for the algorithms in namespace `std`.
- `{last, result + N}`, for the algorithms in namespace `ranges`.

14 *Complexity*: Exactly `last - first` applications of the corresponding predicate and any projection.

15 *Remarks*: Stable ([`algorithm.stable`]).

§ [alg.partitions]

Change [`alg.partitions`] as follows:



```
template<class InputIterator, class Predicate>
constexpr bool is_partitioned(InputIterator first, InputIterator last,
                             Predicate UnaryTest pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
bool is_partitioned(ExecutionPolicy&& exec,
                   ForwardIterator first, ForwardIterator last, Predicate pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         indirect_unary_predicate indirect_unary_test<projected<I, Proj>>
         constexpr bool ranges::is_partitioned(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
         indirect_unary_test<projected<iterator_t<R>, Proj>> Pred>
constexpr bool ranges::is_partitioned(R&& r, Pred pred, Proj proj = {});
```

1 Let `proj` be `identity{}` for the overloads with no parameter named `proj`.

2 *Returns*: `true` if and only if the elements `e` of `[first, last)` are partitioned with respect to the expression `bool(invoker(pred, invoker(proj, e)))`.

3 *Complexity*: Linear. At most `last - first` applications of `pred` and `proj`.

```
template<class ForwardIterator, class Predicate UnaryTest>
constexpr ForwardIterator
partition(ForwardIterator first, ForwardIterator last, Predicate UnaryTest pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator
partition(ExecutionPolicy&& exec,
          ForwardIterator first, ForwardIterator last, Predicate pred);

template<permutable I, sentinel_for<I> S, class Proj = identity,
         indirect_unary_predicate indirect_unary_test<projected<I, Proj>>
         constexpr subrange<I>
         ranges::partition(I first, S last, Pred pred, Proj proj = {});
template<forward_range R, class Proj = identity,
         indirect_unary_predicate indirect_unary_test<projected<iterator_t<R>, Proj>>
         constexpr forward_range
         ranges::partition(R&& r, Pred pred, Proj proj = {});
```



```
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R>
ranges::partition(R&& r, Pred pred, Proj proj = {});
```

4 Let `proj` be `identity{}` for the overloads with no parameter named `proj` and let $E(x)$ be `bool{invoke(pred, invoke(proj, x))}`.

5 *Preconditions*: For the overloads in namespace `std`, `ForwardIterator` meets the `Cpp17ValueSwappable` requirements ([\[swappable.requirements\]](#)).

6 *Effects*: Places all the elements e in $[first, last)$ that satisfy $E(e)$ before all the elements that do not.

7 *Returns*: Let i be an iterator such that $E(*j)$ is `true` for every iterator j in $[first, i)$ and `false` for every iterator j in $[i, last)$. Returns:

- i for the overloads in namespace `std`.
- `{i, last}` for the overloads in namespace `ranges`.

8 *Complexity*: Let $N = last - first$:

- For the overload with no `ExecutionPolicy`, exactly N applications of the predicate and projection. At most $N / 2$ swaps if the type of `first` meets the `Cpp17BidirectionalIterator` requirements for the overloads in namespace `std` or models `bidirectional_iterator` for the overloads in namespace `ranges`, and at most N swaps otherwise.
- For the overload with an `ExecutionPolicy`, $\mathcal{O}(N \log N)$ swaps and $\mathcal{O}(N)$ applications of the predicate.

[...]

```
template<class InputIterator, class OutputIterator1, class OutputIterator2,
        class Predicate UnaryTest>
constexpr pair<OutputIterator1, OutputIterator2>
partition_copy(InputIterator first, InputIterator last,
               OutputIterator1 out_true, OutputIterator2 out_false,
               Predicate UnaryTest pred);
template<class ExecutionPolicy, class ForwardIterator, class ForwardIterator2,
        class ForwardIterator1, class Predicate UnaryTest>
pair<ForwardIterator1, ForwardIterator2>
partition_copy(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last,
               ForwardIterator1 out_true, ForwardIterator2 out_false,
               Predicate UnaryTest pred);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O1, weakly_incrementable O2,
        class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<I, Proj>>
requires indirectly_copyable<I, O1> && indirectly_copyable<I, O2>
constexpr ranges::partition_copy_result<I, O1, O2>
ranges::partition_copy(I first, S last, O1 out_true, O2 out_false, Proj proj = {});
```



```
template<input_range R, weakly_incrementable O1, weakly_incrementable O2,
        class Proj = identity,
        indirect_unary_predicate indirect_unary_test<projected<iterator_t<R>, O1>, O2>
requires indirectly_copyable<iterator_t<R>, O1> &&
        indirectly_copyable<iterator_t<R>, O2>
constexpr ranges::partition_copy_result<borrowed_iterator_t<R>, O1, O2>
ranges::partition_copy(R&& r, O1 out_true, O2 out_false, Pred pred, Proj proj)
```

14 Let `proj` be `identity{}` for the overloads with no parameter named `proj` and let $E(x)$ be `bool{invoke(pred, invoke(proj, x))}`.

15 *Mandates*: For the overloads in namespace `std`, the expression `*first` is writable (`[iterator.requirements.general]`) to `out_true` and `out_false`.

16 *Preconditions*: The input range and output ranges do not overlap.

[Note: For the overload with an `ExecutionPolicy`, there might be a performance cost if `first`'s value type does not meet the *Cpp17CopyConstructible* requirements. — end note]

17 *Effects*: For each iterator `i` in `[first, last)`, copies `*i` to the output range beginning with `out_true` if $E(*i)$ is `true`, or to the output range beginning with `out_false` otherwise.

18 *Returns*: Let `o1` be the end of the output range beginning at `out_true`, and `o2` the end of the output range beginning at `out_false`. Returns

- `{o1, o2}` for the overloads in namespace `std`.
- `{last, o1, o2}` for the overloads in namespace `ranges`.

19 *Complexity*: Exactly `last - first` applications of `pred` and `proj`.

§ 8. References

[N5008] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>